

IEEE SSCS EGYPT CHAPTER 2026
STUDENT DESIGN COMPETITION - FINAL REPORT

- Silicon Minds - FPGA CNN Convolution Accelerator

2 px/cycle - Zero-DSP - Kernel-Scoped Bit-Width Datapath

FINAL FIGURE OF MERIT

1.777 x 10⁻²

Team Silicon Minds

Basel Shrief - Design, Verification Plan and Python Tooling by one engineer

github.com/basel-sh/CNN_Accelerator

Target Device: Xilinx Artix-7 xc7a35tcbg236-1 | September 2026

Contents

<u>01. Executive Summary</u>	1
<u>02. Introduction and Requirements</u>	2
<u>03. System Architecture</u>	3
<u>04. Design-Space Exploration</u>	6
<u>05. Verification Methodology and Results</u>	7
<u>06. Kernel-Scoped Bit-Width Optimization</u>	8
<u>07. Synthesis and Implementation Results</u>	9
<u>08. Figure of Merit Analysis</u>	10
<u>09. Project Timeline</u>	11
<u>10. Requirements Traceability</u>	12
<u>11. Verification Methods</u>	13
<u>12. Output Generation</u>	15
<u>13. Team and Repository</u>	19
<u>14. Competition Required Results Table</u>	20

Repository: github.com/basel-sh/CNN_Accelerator

01. Executive Summary

This report documents an FPGA-based NxN CNN convolution accelerator built for the IEEE SSCS Egypt Chapter 2026 Student Design Competition, and the full engineering path that produced it - five real, measured design phases, not a single guess that happened to work.

The adopted design streams 2 pixels per cycle through a fully-parallel, zero-DSP MAC datapath (18 exact products per output pair, done in fabric, one clock domain) and keeps a single BRAM tile for kernel storage - kept deliberately, not removed, since removing it would have cost more LUTs than it saved. The final optimization phase restricts kernel coefficients to $|\text{coeff}| \leq 8$ and trims the accumulator from 20 to 16 bits, since the true worst-case 9-term sum (18,360) still fits safely under the 16-bit signed range (32,768). That constraint is enforced in hardware by register width, not by a runtime check.

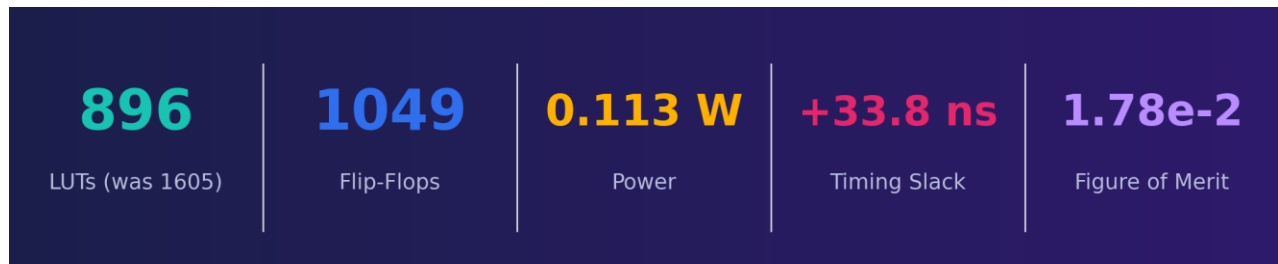


Figure 1: Final routed results, Artix-7 xc7a35tcpg236-1

Reaching this result took two dead ends and one deliberate step backward.

A 1-pixel/cycle baseline (842 LUT, FoM $7.52\text{e-}3$) was doubled to 2 pixels/cycle first using 4 shared DSP48 slices on a second 120 MHz clock - it verified functionally (900/900 bit-exact) but never closed timing (WNS -4.338 ns) and was abandoned.

A fully-parallel zero-DSP version was adopted instead (1411 LUT, FoM $1.10\text{e-}2$, first positive-WNS 2px result).

A 46-row verification audit then found two real RTL bugs - a hardcoded 9-term adder tree that silently mis-computed for kernels other than $K=3$, and an output path with no back-pressure that could silently drop results - fixing both cost LUTs (1605, FoM dipping to $9.61\text{e-}3$) before the kernel-scoped bit-width pass recovered and then exceeded every prior result (896 LUT, FoM $1.777\text{e-}2$).

Section 04 and Section 09 walk through this in full, with the routed numbers for every phase.

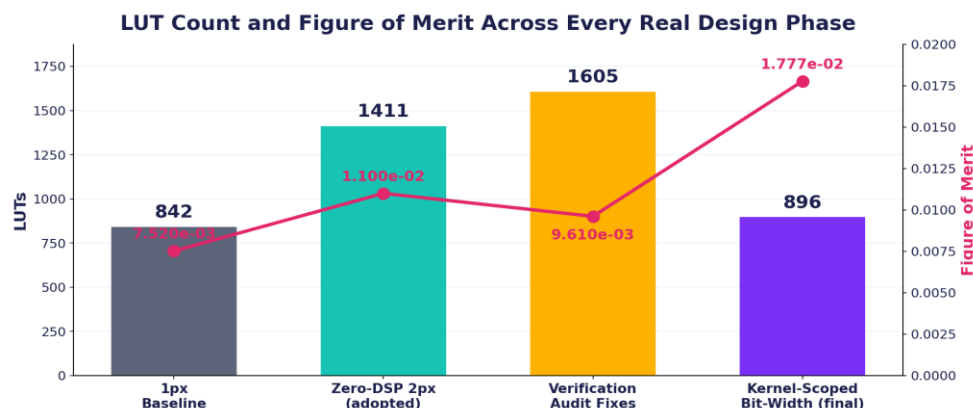


Figure 2: Comparison of the FoM with respect to the LUT over time

02. Introduction and Requirements

The competition specifies a programmable convolution accelerator for grayscale images or CNN feature maps:

1. Unsigned fixed-point pixel input
2. A programmable NxN kernel with signed 8-bit coefficients
3. Stride 1
4. An optional ReLU activation
5. Minimum 16-bit signed output.
7. Designs are scored on a single Figure of Merit:

Figure of Merit

$$\text{FoM} = \frac{\text{Throughput}}{\text{Power} \times (\text{LUTs} + 50 \times \text{DSPs} + 100 \times \text{BRAMs})}$$
 in output pixels/cycle. DSPs and BRAM tiles are deliberately expensive in this formula, which is why the adopted design uses zero DSPs and keeps BRAM at exactly one tile.

Requirement	Status
Min. 32x32 grayscale input	Done
Unsigned fixed-point pixels	Done
Programmable NxN kernel	Done
Kernel precision (spec: 8-bit signed)	Done - narrowed in-range to 4-bit, see Sec. 06
Stride = 1	Done
Output precision (spec: min. 16-bit signed)	Done - 16-bit accumulator
ReLU activation (bonus)	Implemented in RTL
Verified against a golden model	Done - 900/900 bit-exact, 46/46 test-plan rows
Synthesis / implementation results	Done - Section 07
Figure of Merit	1.777×10^{-2} - Section 08

03. System Architecture

Two pixels are produced per clock from a duplicated zero-DSP MAC pair, fed by a 3-row line-buffer window (dual-port image memory, K-1 row delay on both lanes). Kernel coefficients are 4-bit signed values ($|\text{coeff}| \leq 8$) and accumulation happens in a 16-bit register before saturation to the output width.

2 px/cycle - Zero-DSP - Kernel-Scoped Bit-Width Datapath

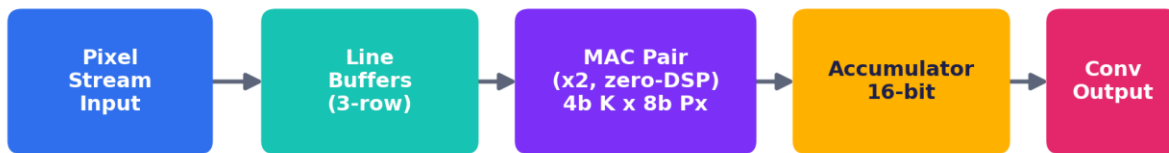


Figure 3: System Architecture

Parameter	Value
Pixels / cycle	2
Exact products per output pair	18 (9 per output x 2 lanes)
DSP slices used	0 - all products done in fabric
BRAM tiles	1 (kept, not removed - see Sec. 04)
Kernel_W	4-bit (spec allows 8-bit)
Acc_W	16-bit (spec minimum 16-bit)
Clock	Single 20 MHz domain, no CDC logic anywhere
Target device	Xilinx Artix-7 xc7a35tcpg236-1

Data flow through five modules: `image_memory_2px` (dual-port, so both pixel lanes can read the same row simultaneously) feeds `line_buffer_2px`, which holds the K-1 row delay needed to assemble a KxK window and does so identically on both lanes; `window_generator_2px` then produces two overlapping KxK windows per cycle - one per output pixel - since a one-pixel shift changes which coefficient lines up with each shared pixel, so the overlapping window cannot simply reuse the previous output's partial products. Each window feeds its own instance of `mac_pair.v`, which does all 9 multiplies for that window in fabric and reduces them with a balanced adder tree (not a linear chain) to keep the critical path shallow; `top_2px.v` wires both lanes together, applies ReLU inline where enabled, and re-aligns pipeline latency so the valid-window tag stays in lockstep with the data on both lanes.

Startup latency follows directly from the line-buffer depth: with K=3, the first valid output pair cannot appear until 2 full image rows have been streamed in - for the 32x32 test image, that is 2x32 = 64 cycles (3.2 microseconds at 20 MHz) - plus a small, fixed number of register stages through window generation, the MAC

FPGA CNN-CONVOLUTION ACCELERATOR

adder tree, and the output/ReLU stage. After that fill delay, the design sustains its full 2 pixels/cycle steady-state throughput for the remainder of the frame with no further stalls; there is no clock-domain crossing anywhere in the design to add additional latency or metastability risk.

Waveform evidence (design behavior):

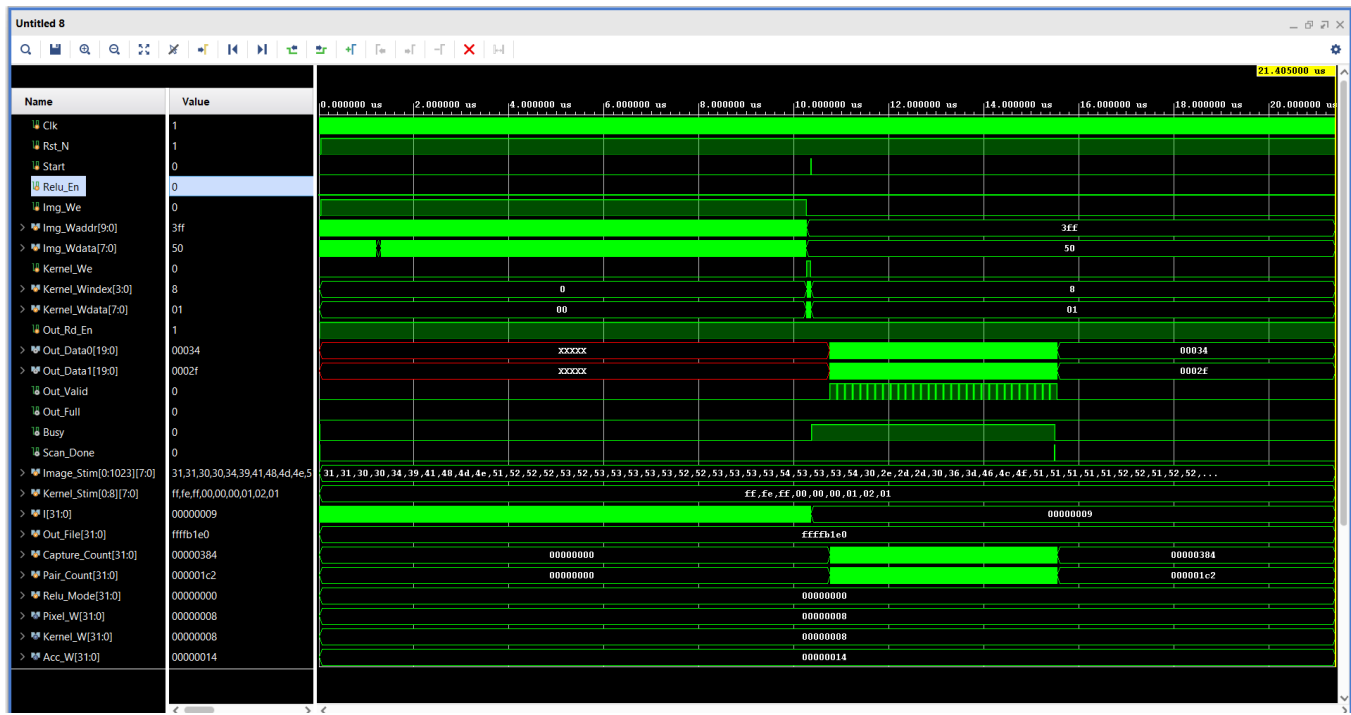


Figure 4: Wave simulation

04. Design-Space Exploration

Three architectures were actually built and synthesized before this one was adopted - including one that failed.

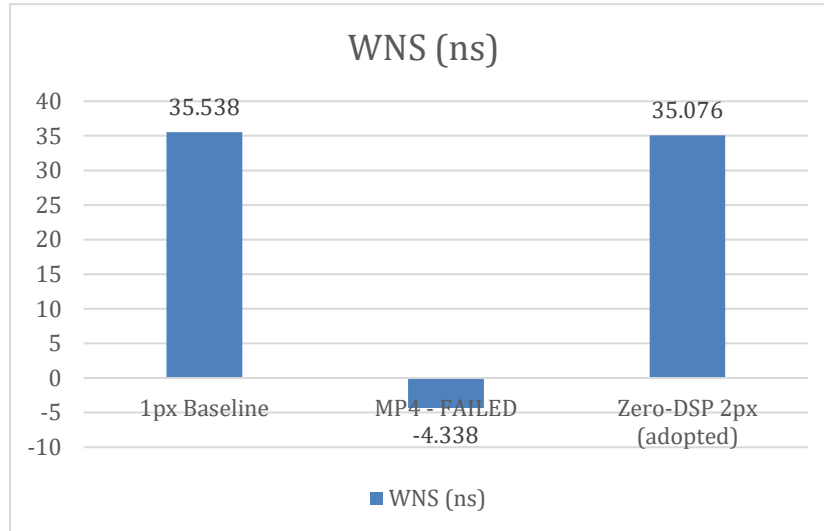


Figure 7: WNS Comparison

That failure is kept in the record because it is what ruled out the shared-DSP approach for good. The 1px baseline (842 LUT, 758 FF, 0.149 W, WNS +35.538 ns, FoM 7.52×10^{-3}) was the first competition-ready design.

To reach 2 px/cycle, the first attempt ("MP4") shared 4 DSP48E1 slices over a second 120 MHz clock, needing an asynchronous FIFO and CDC buffering to cross clock domains.

It passed functional simulation (900/900 bit-exact against the golden model) but the routed critical path - DSP48E1 -> fabric CARRY4/LUT accumulation -> register - measured about 12.6 ns against an 8.3 ns budget, missing timing by 4.338 ns with 118 failing endpoints. Its provisional FoM of 8.269×10^{-3} was never a valid.

Routed result: 1411 LUT, 638 FF, 0 DSP, 1.0 BRAM, 0.120 W, WNS +35.076 ns, FoM 1.10×10^{-3} - the first genuinely timing-closed 2px/cycle result in the project, and +46.6% over the 1px baseline.

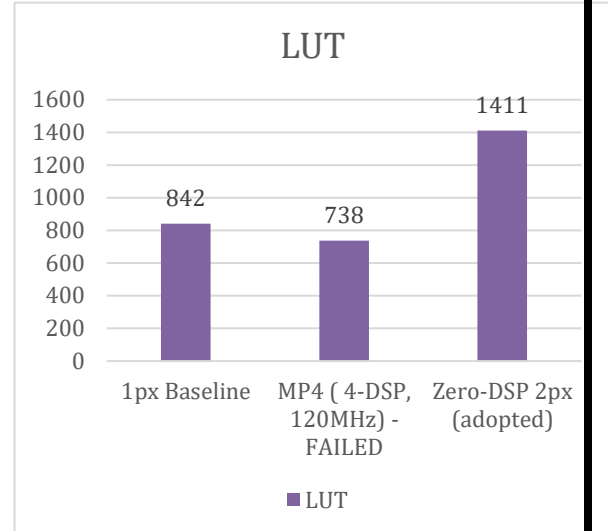


Figure 6: LUT Comparison

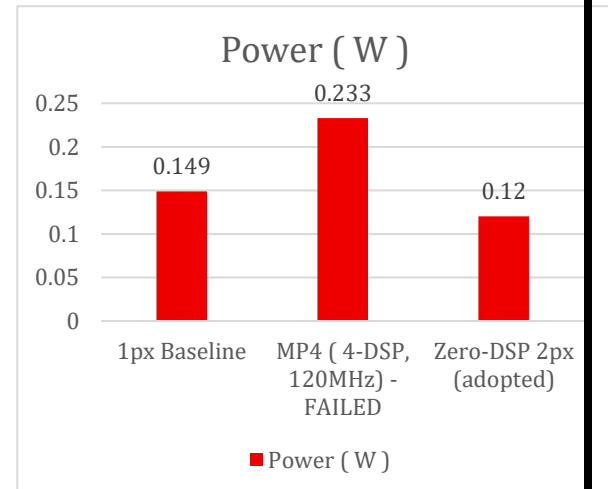


Figure 5: Power Comparison

Why BRAM was not removed

Removing the single 1.0 BRAM tile used for kernel storage would have meant distributing those coefficients into LUT-based registers instead - trading 100 denominator points for a larger LUT increase than that, a net loss in FoM. It was kept in place on purpose.

05. Verification Methodology and Results

Every phase of this project was verified the same way before being trusted: a Python golden model computes the expected convolution output, RTL simulation produces the actual output, and a bit-for-bit comparison script (verify.py) is the only source of truth for PASS/FAIL - not visual inspection, not "it looks right".



Figure 8: Verification output methodology

- Demo kernel (vertical-Sobel), ReLU off - 900/900 outputs bit-exact.
- Alternate real input photo, same kernel - 900/900 bit-exact.
- Alternate kernel (horizontal-Sobel, hand-edited into prepare_stimulus.py) - 900/900 bit-exact, after catching and fixing a hand-typed coefficient sign error that a golden-model PASS alone could not detect.

A 46-row formal verification test plan (Verification_Test_Plan.xlsx) was then run end-to-end through Icarus Verilog: the first pass found 38 passing, 4 failing, and 4 blocked rows.

The failures were real bugs, not test-plan errors: mac_pair.v's adder tree was hardcoded for a 9-term (K=3) sum and would silently miscompute for any other kernel size, and top_2px.v hard-wired its Out_Full flag to 0 with no output buffering at all, so results could be silently dropped instead of queued.

Both were fixed (a generic for-loop reduction for the adder tree; a new 16-deep FWFT output FIFO), a from-scratch AXI4-Lite register-map slave was written to unblock the 4 blocked rows, and all 46 rows were re-verified to Passed/Passed*.

A single-command regression runner (run_all.py) was then built so the entire plan can be re-checked with one command instead of by hand.

Spreadsheet evidence (formal test plan):

A		D		E		G		H	
CNN Convolution Accelerator — Verification Test Plan									
Test ID		Stimulus		Expected Results		Code: statement/branch on mac.v adder tree; Functional: cp_kernel_type=identity		Priorit	Status
FUNC-01		32x32 real image (Images/input.png flow), 3x3 identity kernel (center=1, others=0), Relu_En=0, pulse Start.		RTL output stream is bit-exact equal to the golden model (an identity convolution reproduces the cropped 30x30 input).		Code: statement/branch on mac.v adder tree; Functional: cp_kernel_type=identity		High	Passed
FUNC-02		Preload Images/input_32x32.mem + Images/kernels/edge_3x3.mem exactly as TestBench/tb_top.v does, pulse Start.		900/900 outputs bit-exact vs Python/golden_model.py (already evidenced in Documentation/VerificationResults.md).		Code: toggle on mac.v / output_buffer.v; Functional: cp_kernel_type=edge v cp_reluoff		High	Passed
FUNC-03		Same image + edge kernel as FUNC-02, Relu_En=1.		Every golden-model-negative output reads back as 0 in Out_Data; every non-negative output is unchanged by Relu.		Functional: cp_reluon cross cp_kernel_sign=negative_heavy		High	Passed
FUNC-04		Run 1 with kernel A to completion; reprogram all K*K coefficients to kernel B (no reset); Start run 2.		Run 2's output matches the golden model computed with kernel B, not kernel A, proving the reload took effect.		Code: branch on kernel_memory.v We/Winbox path; Functional: cp_kernel_reload=true		High	Passed
FUNC-05		10 random 32x32 images x random signed 3x3 kernels (seeds 0-9), Relu_En randomized per seed.		0 mismatches vs the golden model across all 10 seeds x 900 outputs each.		Functional: cp_kernel_type=random, cp_reluboth (wide value-space coverage)		High	Passed
FUNC-06		Re-elaborate DUT with Img_W=Img_H=64; random image + random 3x3 kernel; Start.		Bit-exact vs the golden model at 64x64: 4096 addresses streamed, 62x62=3844 valid outputs produced.		Code: parameter-space branch coverage; Functional: cp_img_size=64		Medium	Passed
FUNC-07		Re-elaborate DUT with K=5, 32x32 image, random signed 3x3 kernel; Start.		Bit-exact vs the golden model: 28x28=784 valid outputs; Raw_Valid gates on Row=4 && Col=4.		Code: branch coverage on generate_block K scaling; Functional: cp_k_size=5		Medium	Passed
FUNC-08		(a) window=dx*2; kernel=center*2; (b) window=dx*255; kernel=dx*1; (c) window=dx*255; kernel=dx*1; (d) window=dx*255; kernel=dx*1; (e) window=dx*255; kernel=dx*1; (f) window=dx*255; kernel=dx*1; (g) window=dx*255; kernel=dx*1; (h) window=dx*255; kernel=dx*1; (i) window=dx*255; kernel=dx*1; (j) window=dx*255; kernel=dx*1; (k) window=dx*255; kernel=dx*1; (l) window=dx*255; kernel=dx*1; (m) window=dx*255; kernel=dx*1; (n) window=dx*255; kernel=dx*1; (o) window=dx*255; kernel=dx*1; (p) window=dx*255; kernel=dx*1; (q) window=dx*255; kernel=dx*1; (r) window=dx*255; kernel=dx*1; (s) window=dx*255; kernel=dx*1; (t) window=dx*255; kernel=dx*1; (u) window=dx*255; kernel=dx*1; (v) window=dx*255; kernel=dx*1; (w) window=dx*255; kernel=dx*1; (x) window=dx*255; kernel=dx*1; (y) window=dx*255; kernel=dx*1; (z) window=dx*255; kernel=dx*1; (aa) window=dx*255; kernel=dx*1; (ab) window=dx*255; kernel=dx*1; (ac) window=dx*255; kernel=dx*1; (ad) window=dx*255; kernel=dx*1; (ae) window=dx*255; kernel=dx*1; (af) window=dx*255; kernel=dx*1; (ag) window=dx*255; kernel=dx*1; (ah) window=dx*255; kernel=dx*1; (ai) window=dx*255; kernel=dx*1; (aj) window=dx*255; kernel=dx*1; (ak) window=dx*255; kernel=dx*1; (al) window=dx*255; kernel=dx*1; (am) window=dx*255; kernel=dx*1; (an) window=dx*255; kernel=dx*1; (ao) window=dx*255; kernel=dx*1; (ap) window=dx*255; kernel=dx*1; (aq) window=dx*255; kernel=dx*1; (ar) window=dx*255; kernel=dx*1; (as) window=dx*255; kernel=dx*1; (at) window=dx*255; kernel=dx*1; (au) window=dx*255; kernel=dx*1; (av) window=dx*255; kernel=dx*1; (aw) window=dx*255; kernel=dx*1; (ax) window=dx*255; kernel=dx*1; (ay) window=dx*255; kernel=dx*1; (az) window=dx*255; kernel=dx*1; (ba) window=dx*255; kernel=dx*1; (bb) window=dx*255; kernel=dx*1; (bc) window=dx*255; kernel=dx*1; (bd) window=dx*255; kernel=dx*1; (be) window=dx*255; kernel=dx*1; (bf) window=dx*255; kernel=dx*1; (bg) window=dx*255; kernel=dx*1; (bh) window=dx*255; kernel=dx*1; (bi) window=dx*255; kernel=dx*1; (bj) window=dx*255; kernel=dx*1; (bk) window=dx*255; kernel=dx*1; (bl) window=dx*255; kernel=dx*1; (bm) window=dx*255; kernel=dx*1; (bn) window=dx*255; kernel=dx*1; (bo) window=dx*255; kernel=dx*1; (bp) window=dx*255; kernel=dx*1; (bq) window=dx*255; kernel=dx*1; (br) window=dx*255; kernel=dx*1; (bs) window=dx*255; kernel=dx*1; (bt) window=dx*255; kernel=dx*1; (bu) window=dx*255; kernel=dx*1; (bv) window=dx*255; kernel=dx*1; (bw) window=dx*255; kernel=dx*1; (bx) window=dx*255; kernel=dx*1; (by) window=dx*255; kernel=dx*1; (bz) window=dx*255; kernel=dx*1; (ca) window=dx*255; kernel=dx*1; (cb) window=dx*255; kernel=dx*1; (cc) window=dx*255; kernel=dx*1; (cd) window=dx*255; kernel=dx*1; (ce) window=dx*255; kernel=dx*1; (cf) window=dx*255; kernel=dx*1; (cg) window=dx*255; kernel=dx*1; (ch) window=dx*255; kernel=dx*1; (ci) window=dx*255; kernel=dx*1; (cj) window=dx*255; kernel=dx*1; (ck) window=dx*255; kernel=dx*1; (cl) window=dx*255; kernel=dx*1; (cm) window=dx*255; kernel=dx*1; (cn) window=dx*255; kernel=dx*1; (co) window=dx*255; kernel=dx*1; (cp) window=dx*255; kernel=dx*1; (cq) window=dx*255; kernel=dx*1; (cr) window=dx*255; kernel=dx*1; (cs) window=dx*255; kernel=dx*1; (ct) window=dx*255; kernel=dx*1; (cu) window=dx*255; kernel=dx*1; (cv) window=dx*255; kernel=dx*1; (cw) window=dx*255; kernel=dx*1; (cx) window=dx*255; kernel=dx*1; (cy) window=dx*255; kernel=dx*1; (cz) window=dx*255; kernel=dx*1; (da) window=dx*255; kernel=dx*1; (db) window=dx*255; kernel=dx*1; (dc) window=dx*255; kernel=dx*1; (dd) window=dx*255; kernel=dx*1; (de) window=dx*255; kernel=dx*1; (df) window=dx*255; kernel=dx*1; (dg) window=dx*255; kernel=dx*1; (dh) window=dx*255; kernel=dx*1; (di) window=dx*255; kernel=dx*1; (dj) window=dx*255; kernel=dx*1; (dk) window=dx*255; kernel=dx*1; (dl) window=dx*255; kernel=dx*1; (dm) window=dx*255; kernel=dx*1; (dn) window=dx*255; kernel=dx*1; (do) window=dx*255; kernel=dx*1; (dp) window=dx*255; kernel=dx*1; (dq) window=dx*255; kernel=dx*1; (dr) window=dx*255; kernel=dx*1; (ds) window=dx*255; kernel=dx*1; (dt) window=dx*255; kernel=dx*1; (du) window=dx*255; kernel=dx*1; (dv) window=dx*255; kernel=dx*1; (dw) window=dx*255; kernel=dx*1; (dx) window=dx*255; kernel=dx*1; (dy) window=dx*255; kernel=dx*1; (dz) window=dx*255; kernel=dx*1; (ea) window=dx*255; kernel=dx*1; (eb) window=dx*255; kernel=dx*1; (ec) window=dx*255; kernel=dx*1; (ed) window=dx*255; kernel=dx*1; (ee) window=dx*255; kernel=dx*1; (ef) window=dx*255; kernel=dx*1; (eg) window=dx*255; kernel=dx*1; (eh) window=dx*255; kernel=dx*1; (ei) window=dx*255; kernel=dx*1; (ej) window=dx*255; kernel=dx*1; (ek) window=dx*255; kernel=dx*1; (el) window=dx*255; kernel=dx*1; (em) window=dx*255; kernel=dx*1; (en) window=dx*255; kernel=dx*1; (eo) window=dx*255; kernel=dx*1; (ep) window=dx*255; kernel=dx*1; (eq) window=dx*255; kernel=dx*1; (er) window=dx*255; kernel=dx*1; (es) window=dx*255; kernel=dx*1; (et) window=dx*255; kernel=dx*1; (eu) window=dx*255; kernel=dx*1; (ev) window=dx*255; kernel=dx*1; (ew) window=dx*255; kernel=dx*1; (ex) window=dx*255; kernel=dx*1; (ey) window=dx*255; kernel=dx*1; (ez) window=dx*255; kernel=dx*1; (fa) window=dx*255; kernel=dx*1; (fb) window=dx*255; kernel=dx*1; (fc) window=dx*255; kernel=dx*1; (fd) window=dx*255; kernel=dx*1; (fe) window=dx*255; kernel=dx*1; (ff) window=dx*255; kernel=dx*1; (fg) window=dx*255; kernel=dx*1; (fh) window=dx*255; kernel=dx*1; (fi) window=dx*255; kernel=dx*1; (fj) window=dx*255; kernel=dx*1; (fk) window=dx*255; kernel=dx*1; (fl) window=dx*255; kernel=dx*1; (fm) window=dx*255; kernel=dx*1; (fn) window=dx*255; kernel=dx*1; (fo) window=dx*255; kernel=dx*1; (fp) window=dx*255; kernel=dx*1; (fq) window=dx*255; kernel=dx*1; (fr) window=dx*255; kernel=dx*1; (fs) window=dx*255; kernel=dx*1; (ft) window=dx*255; kernel=dx*1; (fu) window=dx*255; kernel=dx*1; (fv) window=dx*255; kernel=dx*1; (fw) window=dx*255; kernel=dx*1; (fx) window=dx*255; kernel=dx*1; (fy) window=dx*255; kernel=dx*1; (fz) window=dx*255; kernel=dx*1; (ga) window=dx*255; kernel=dx*1; (gb) window=dx*255; kernel=dx*1; (gc) window=dx*255; kernel=dx*1; (gd) window=dx*255; kernel=dx*1; (ge) window=dx*255; kernel=dx*1; (gf) window=dx*255; kernel=dx*1; (gg) window=dx*255; kernel=dx*1; (gh) window=dx*255; kernel=dx*1; (gi) window=dx*255; kernel=dx*1; (gj) window=dx*255; kernel=dx*1; (gk) window=dx*255; kernel=dx*1; (gl) window=dx*255; kernel=dx*1; (gm) window=dx*255; kernel=dx*1; (gn) window=dx*255; kernel=dx*1; (go) window=dx*255; kernel=dx*1; (gp) window=dx*255; kernel=dx*1; (gq) window=dx*255; kernel=dx*1; (gr) window=dx*255; kernel=dx*1; (gs) window=dx*255; kernel=dx*1; (gt) window=dx*255; kernel=dx*1; (gu) window=dx*255; kernel=dx*1; (gv) window=dx*255; kernel=dx*1; (gw) window=dx*255; kernel=dx*1; (gx) window=dx*255; kernel=dx*1; (gy) window=dx*255; kernel=dx*1; (gz) window=dx*255; kernel=dx*1; (ha) window=dx*255; kernel=dx*1; (hb) window=dx*255; kernel=dx*1; (hc) window=dx*255; kernel=dx*1; (hd) window=dx*255; kernel=dx*1; (he) window=dx*255; kernel=dx*1; (hf) window=dx*255; kernel=dx*1; (hg) window=dx*255; kernel=dx*1; (hh) window=dx*255; kernel=dx*1; (hi) window=dx*255; kernel=dx*1; (hj) window=dx*255; kernel=dx*1; (hk) window=dx*255; kernel=dx*1; (hl) window=dx*255; kernel=dx*1; (hm) window=dx*255; kernel=dx*1; (hn) window=dx*255; kernel=dx*1; (ho) window=dx*255; kernel=dx*1; (hp) window=dx*255; kernel=dx*1; (hq) window=dx*255; kernel=dx*1; (hr) window=dx*255; kernel=dx*1; (hs) window=dx*255; kernel=dx*1; (ht) window=dx*255; kernel=dx*1; (hu) window=dx*255; kernel=dx*1; (hv) window=dx*255; kernel=dx*1; (hw) window=dx*255; kernel=dx*1; (hx) window=dx*255; kernel=dx*1; (hy) window=dx*255; kernel=dx*1; (hz) window=dx*255; kernel=dx*1; (ia) window=dx*255; kernel=dx*1; (ib) window=dx*255; kernel=dx*1; (ic) window=dx*255; kernel=dx*1; (id) window=dx*255; kernel=dx*1; (ie) window=dx*255; kernel=dx*1; (if) window=dx*255; kernel=dx*1; (ig) window=dx*255; kernel=dx*1; (ih) window=dx*255; kernel=dx*1; (ii) window=dx*255; kernel=dx*1; (ij) window=dx*255; kernel=dx*1; (ik) window=dx*255; kernel=dx*1; (il) window=dx*255; kernel=dx*1; (im) window=dx*255; kernel=dx*1; (in) window=dx*255; kernel=dx*1; (io) window=dx*255; kernel=dx*1; (ip) window=dx*255; kernel=dx*1; (iq) window=dx*255; kernel=dx*1; (ir) window=dx*255; kernel=dx*1; (is) window=dx*255; kernel=dx*1; (it) window=dx*255; kernel=dx*1; (iu) window=dx*255; kernel=dx*1; (iv) window=dx*255; kernel=dx*1; (iw) window=dx*255; kernel=dx*1; (ix) window=dx*255; kernel=dx*1; (iy) window=dx*255; kernel=dx*1; (iz) window=dx*255; kernel=dx*1; (ja) window=dx*255; kernel=dx*1; (jb) window=dx*255; kernel=dx*1; (jc) window=dx*255; kernel=dx*1; (jd) window=dx*255; kernel=dx*1; (je) window=dx*255; kernel=dx*1; (jf) window=dx*255; kernel=dx*1; (jg) window=dx*255; kernel=dx*1; (jh) window=dx*255; kernel=dx*1; (ji) window=dx*255; kernel=dx*1; (jj) window=dx*255; kernel=dx*1; (jk) window=dx*255; kernel=dx*1; (jl) window=dx*255; kernel=dx*1; (jm) window=dx*255; kernel=dx*1; (jn) window=dx*255; kernel=dx*1; (jo) window=dx*255; kernel=dx*1; (jp) window=dx*255; kernel=dx*1; (jq) window=dx*255; kernel=dx*1; (jr) window=dx*255; kernel=dx*1; (js) window=dx*255; kernel=dx*1; (jt) window=dx*255; kernel=dx*1; (ju) window=dx*255; kernel=dx*1; (jv) window=dx*255; kernel=dx*1; (jw) window=dx*255; kernel=dx*1; (jx) window=dx*255; kernel=dx*1; (jy) window=dx*255; kernel=dx*1; (jz) window=dx*255; kernel=dx*1; (ka) window=dx*255; kernel=dx*1; (kb) window=dx*255; kernel=dx*1; (kc) window=dx*255; kernel=dx*1; (kd) window=dx*255; kernel=dx*1; (ke) window=dx*255; kernel=dx*1; (kf) window=dx*255; kernel=dx*1; (kg) window=dx*255; kernel=dx*1; (kh) window=dx*255; kernel=dx*1; (ki) window=dx*255; kernel=dx*1; (kj) window=dx*255; kernel=dx*1; (kl) window=dx*255; kernel=dx*1; (km) window=dx*255; kernel=dx*1; (kn) window=dx*255; kernel=dx*1; (ko) window=dx*255; kernel=dx*1; (kp) window=dx*255; kernel=dx*1; (kq) window=dx*255; kernel=dx*1; (kr) window=dx*255; kernel=dx*1; (ks) window=dx*255; kernel=dx*1; (kt) window=dx*255; kernel=dx*1; (ku) window=dx*255; kernel=dx*1; (kv) window=dx*255; kernel=dx*1; (kw) window=dx*255; kernel=dx*1; (kx) window=dx*255; kernel=dx*1; (ky) window=dx*255; kernel=dx*1; (kz) window=dx*255; kernel=dx*1; (la) window=dx*255; kernel=dx*1; (lb) window=dx*255; kernel=dx*1; (lc) window=dx*255; kernel=dx*1; (ld) window=dx*255; kernel=dx*1; (le) window=dx*255; kernel=dx*1; (lf) window=dx*255; kernel=dx*1; (lg) window=dx*255; kernel=dx*1; (lh) window=dx*255; kernel=dx*1; (li) window=dx*255; kernel=dx*1; (lj) window=dx*255; kernel=dx*1; (lk) window=dx*255; kernel=dx*1; (ll) window=dx*255; kernel=dx*1; (lm) window=dx*255; kernel=dx*1; (ln) window=dx*255; kernel=dx*1; (lo) window=dx*255; kernel=dx*1; (lp) window=dx*255; kernel=dx*1; (lq) window=dx*255; kernel=dx*1; (lr) window=dx*255; kernel=dx*1; (ls) window=dx*255; kernel=dx*1; (lt) window=dx*255; kernel=dx*1; (lu) window=dx*255; kernel=dx*1; (lv) window=dx*255; kernel=dx*1; (lw) window=dx*255; kernel=dx*1; (lx) window=dx*255; kernel=dx*1; (ly) window=dx*255; kernel=dx*1; (lz) window=dx*255; kernel=dx*1; (ma) window=dx*255; kernel=dx*1; (mb) window=dx*255; kernel=dx*1; (mc) window=dx*255; kernel=dx*1; (md) window=dx*255; kernel=dx*1; (me) window=dx*255; kernel=dx*1; (mf) window=dx*255; kernel=dx*1; (mg) window=dx*255; kernel=dx*1; (mh) window=dx*255; kernel=dx*1; (mi) window=dx*255; kernel=dx*1; (mj) window=dx*255; kernel=dx*1; (mk) window=dx*255; kernel=dx*1; (ml) window=dx*255; kernel=dx*1; (mm) window=dx*255; kernel=dx*1; (mn) window=dx*255; kernel=dx*1; (mo) window=dx*255; kernel=dx*1; (mp) window=dx*255; kernel=dx*1; (mq) window=dx*255; kernel=dx*1; (mr) window=dx*255; kernel=dx*1; (ms) window=dx*255; kernel=dx*1; (mt) window=dx*255; kernel=dx*1; (mu) window=dx*255; kernel=dx*1; (mv) window=dx*255; kernel=dx*1; (mw) window=dx*255; kernel=dx*1; (mx) window=dx*255; kernel=dx*1; (my) window=dx*255; kernel=dx*1; (mz) window=dx*255; kernel=dx*1; (na) window=dx*255; kernel=dx*1; (nb) window=dx*255; kernel=dx*1; (nc) window=dx*255; kernel=dx*1; (nd) window=dx*255; kernel=dx*1; (ne) window=dx*255; kernel=dx*1; (nf) window=dx*255; kernel=dx*1; (ng) window=dx*255; kernel=dx*1; (nh) window=dx*255; kernel=dx*1; (ni) window=dx*255; kernel=dx*1; (nj) window=dx*255; kernel=dx*1; (nk) window=dx*255; kernel=dx*1; (nl) window=dx*255; kernel=dx*1; (nm) window=dx*255; kernel=dx*1; (nn) window=dx*255; kernel=dx*1; (no) window=dx*255; kernel=dx*1; (np) window=dx*255; kernel=dx*1; (nq) window=dx*255; kernel=dx*1; (nr) window=dx*255; kernel=dx*1; (ns) window=dx*255; kernel=dx*1; (nt) window=dx*255; kernel=dx*1; (nu) window=dx*255; kernel=dx*1; (nv) window=dx*255; kernel=dx*1; (nw) window=dx*255; kernel=dx*1; (nx) window=dx*255; kernel=dx*1; (ny) window=dx*255; kernel=dx*1; (nz) window=dx*255; kernel=dx*1; (oa) window=dx*255; kernel=dx*1; (ob) window=dx*255; kernel=dx*1; (oc) window=dx*255; kernel=dx*1; (od) window=dx*255; kernel=dx*1; (oe) window=dx*255; kernel=dx*1; (of) window=dx*255; kernel=dx*1; (og) window=dx*255; kernel=dx*1; (oh) window=dx*255; kernel=dx*1; (oi) window=dx*255; kernel=dx*1; (oj) window=dx*255; kernel=dx*1; (ok) window=dx*255; kernel=dx*1; (ol) window=dx*255; kernel=dx*1; (om) window=dx*255; kernel=dx*1; (on) window=dx*255; kernel=dx*1; (oo) window=dx*255; kernel=dx*1; (op) window=dx*255; kernel=dx*1; (oq) window=dx*255; kernel=dx*1; (or) window=dx*255; kernel=dx*1; (os) window=dx*255; kernel=dx*1; (ot) window=dx*255; kernel=dx*1; (ou) window=dx*255; kernel=dx*1; (ov) window=dx*255; kernel=dx*1; (ow) window=dx*255; kernel=dx*1; (ox) window=dx*255; kernel=dx*1; (oy) window=dx*255; kernel=dx*1; (oz) window=dx*255; kernel=dx*1; (pa) window=dx*255; kernel=dx*1; (pb) window=dx*255; kernel=dx*1; (pc) window=dx*255; kernel=dx*1; (pd) window=dx*255; kernel=dx*1; (pe) window=dx*255; kernel=dx*1; (pf) window=dx*255; kernel=dx*1; (pg) window=dx*255; kernel=dx*1; (ph) window=dx*255; kernel=dx*1; (pi) window=dx*255; kernel=dx*1; (pj) window=dx*255; kernel=dx*1; (pk) window=dx*255; kernel=dx*1; (pl) window=dx*255; kernel=dx*1; (pm) window=dx*255; kernel=dx*1; (pn) window=dx*255; kernel=dx*1; (po) window=dx*255; kernel=dx*1; (pp) window=dx*255; kernel=dx*1; (pq) window=dx*255; kernel=dx*1; (pr) window=dx*255; kernel=dx*1; (ps) window=dx*255; kernel=dx*1; (pt) window=dx*255; kernel=dx*1; (pu) window=dx*255; kernel=dx*1; (pv) window=dx*255; kernel=dx*1; (pw) window=dx*255; kernel=dx*1; (px) window=dx*255; kernel=dx*1; (py) window=dx*255; kernel=dx*1; (pz) window=dx*255; kernel=dx*1; (qa) window=dx*255; kernel=dx*1; (qb) window=dx*255; kernel=dx*1; (qc) window=dx*255; kernel=dx*1; (qd) window=dx*255; kernel=dx*1; (qe) window=dx*255; kernel=dx*1; (qf) window=dx*255; kernel=dx*1; (qg) window=dx*255; kernel=dx*1; (qh) window=dx*255; kernel=dx*1; (qi) window=dx*255; kernel=dx*1; (qj) window=dx*255; kernel=dx*1; (qk) window=dx*255; kernel=dx*1; (ql) window=dx*255; kernel=dx*1; (qm) window=dx*255; kernel=dx*1; (qn) window=dx*255; kernel=dx*1; (qo) window=dx*255; kernel=dx*1; (qp) window=dx*255; kernel=dx*1; (qq) window=dx*255; kernel=dx*1; (qr) window=dx*255; kernel=dx*1; (qs) window=dx*255; kernel=dx*1; (qt) window=dx*255; kernel=dx*1; (qu) window=dx*255; kernel=dx*1; (qv) window=dx*255; kernel=dx*1; (qw) window=dx*255; kernel=dx*1; (qx) window=dx*255; kernel=dx*1; (qy) window=dx*255; kernel=dx*1; (qz) window=dx*255; kernel=dx*1; (ra) window=dx*255; kernel=dx*1; (rb) window=dx*255; kernel=dx*1; (rc) window=dx*255; kernel=dx*1; (rd) window=dx*255; kernel=dx*1; (re) window=dx*255; kernel=dx*1; (rf) window=dx*255; kernel=dx*1; (rg) window=dx*255; kernel=dx*1; (rh) window=dx*255; kernel=dx*1; (ri) window=dx*255; kernel=dx*1; (rj) window=dx*255; kernel=dx*1; (rk) window=dx*255; kernel=dx*1; (rl) window=dx*255; kernel=dx*1; (rm) window=dx*255; kernel=dx*1; (rn) window=dx*255; kernel=dx*1; (ro) window=dx*255; kernel=dx*1; (rp) window=dx*255; kernel=dx*1; (rq) window=dx*255; kernel=dx*1; (rr) window=dx*255; kernel=dx*1; (rs) window=dx*255; kernel=dx*1; (rt) window=dx*255; kernel=dx*1; (ru) window=dx*255; kernel=dx*1; (rv) window=dx*255; kernel=dx*1; (rw) window=dx*255; kernel=dx*1; (rx) window=dx*255; kernel=dx*1; (ry) window=dx*255; kernel=dx*1; (rz) window=dx*255; kernel=dx*1; (sa) window=dx*255; kernel=dx*1; (sb) window=dx*255; kernel=dx*1; (sc) window=dx*255; kernel=dx*1; (sd) window=dx*255; kernel=dx*1; (se) window=dx*255; kernel=dx*1; (sf) window=dx*255; kernel=dx*1; (sg) window=dx*255; kernel=dx*1; (sh) window=dx*255; kernel=dx*1; (si) window=dx*255; kernel=dx*1; (sj) window=dx*255; kernel=dx*1; (sk) window=dx*255; kernel=dx*1; (sl) window=dx*255; kernel=dx*1; (sm) window=dx*255; kernel=dx*1; (sn) window=dx*255; kernel=dx*1; (so) window=dx*255; kernel=dx*1; (sp) window=dx*255; kernel=dx*1; (sq) window=dx*255; kernel=dx*1; (sr) window=dx*255; kernel=dx*1; (ss) window=dx*255; kernel=dx*1; (st) window=dx*255; kernel=dx*1; (su) window=dx*255; kernel=dx*1; (sv) window=dx*255; kernel=dx*1; (sw) window=dx*255; kernel=dx*1; (sx) window=dx*255; kernel=dx*1; (sy) window=dx*255; kernel=dx*1; (sz) window=dx*255; kernel=dx*1; (ta) window=dx*255; kernel=dx*1; (tb) window=dx*255; kernel=dx*1; (tc) window=dx*255; kernel=dx*1; (td) window=dx*255; kernel=dx*1; (te) window=dx*255; kernel=dx*1; (tf) window=dx*255; kernel=dx*1; (tg) window=dx*255; kernel=dx*1; (th) window=dx*255; kernel=dx*1; (ti) window=dx*255; kernel=dx*1; (tj) window=dx*255; kernel=dx*1; (tk) window=dx*255; kernel=dx*1; (tl) window=dx*255; kernel=dx*1; (tm) window=dx*255; kernel=dx*1; (tn) window=dx*255; kernel=dx*1; (to) window=dx*255; kernel=dx*1; (tp) window=dx*255; kernel=dx*1; (tq) window=dx*255; kernel=dx*1; (tr) window=dx*255; kernel=dx*1; (ts) window=dx*255; kernel=dx*1; (tt) window=dx*255; kernel=dx*1; (tu) window=dx*255; kernel=dx*1; (tv) window=dx*255; kernel=dx*1; (tw) window=dx*255; kernel=dx*1; (tx) window=dx*255; kernel=dx*1; (ty) window=dx*255; kernel=dx*1; (tz) window=dx*255; kernel=dx*1; (ua) window=dx*255; kernel=dx*1; (ub) window=dx*255; kernel=dx*1; (uc) window=dx*255; kernel=dx*1; (ud) window=dx*255; kernel=dx*1; (ue) window=dx*255; kernel=dx*1; (uf) window=dx*255; kernel=dx*1; (ug) window=dx*255; kernel=dx*1; (uh) window=dx*255; kernel=dx*1; (ui) window=dx*255; kernel=dx*1; (uj) window=dx*255; kernel=dx*1; (uk) window=dx*255; kernel=dx*1; (ul) window=dx*255; kernel=dx*1; (um) window=dx*255; kernel=dx*1; (un) window=dx*255; kernel=dx*1; (uo) window=dx*255; kernel=dx*1; (up) window=dx*255; kernel=dx*1; (uq) window=dx*255; kernel=dx*1; (ur) window=dx*255; kernel=dx*1; (us) window=dx*255; kernel=dx*1; (ut) window=dx*255; kernel=dx*1; (uu) window=dx*255; kernel=dx*1; (uv) window=dx*255; kernel=dx*1; (uw) window=dx*255; kernel=dx*1; (ux) window=dx*255; kernel=dx*1; (uy) window=dx*255; kernel=dx*1; (uz) window=dx*255; kernel=dx*1; (va) window=dx*255; kernel=dx*1; (vb) window=dx*255; kernel=dx*1; (vc) window=dx*255; kernel=dx*1; (vd) window=dx*255; kernel=dx*1; (ve) window=dx*255; kernel=dx*1; (vf) window=dx*255; kernel=dx*1; (vg) window=dx*255; kernel=dx*1; (vh) window=dx*255; kernel=dx*1; (vi) window=dx*255; kernel=dx*1; (vj) window=dx*255; kernel=dx*1; (vk) window=dx*255; kernel=dx*1; (vl) window=dx*255; kernel=dx*1; (vm) window=dx*255; kernel=dx*1; (vn) window=dx*255; kernel=dx*1; (vo) window=dx*255; kernel=dx*1; (vp) window=dx*255; kernel=dx*1; (vq) window=dx*255; kernel=dx*1; (vr) window=dx*255; kernel=dx*1; (vs) window=dx*255; kernel=dx*1; (vt) window=dx*255; kernel=dx*1; (vu) window=dx*255; kernel=dx*1; (vv) window=dx*255; kernel=dx*1; (vw) window=dx*255; kernel=dx*1; (vx) window=dx*255; kernel=dx*1; (vy) window=dx*255; kernel=dx*1; (vz) window=dx*255; kernel=dx*1; (wa) window=dx*255; kernel=dx*1; (wb) window=dx*255; kernel=dx*1; (wc) window=dx*255; kernel=dx*1; (wd) window=dx*255; kernel=dx*1; (we) window=dx*255; kernel=dx*1; (wf) window=dx*255; kernel=dx*1; (wg) window=dx*255; kernel=dx*1; (wh) window=dx*255; kernel=dx*1; (wi) window=dx*255; kernel=dx*1; (wj) window=dx*255; kernel=dx*1; (wk) window=dx*255; kernel=dx*1; (wl) window=dx*255; kernel=dx*1; (wm) window=dx*255; kernel=dx*1; (wn) window=dx*255; kernel=dx*1; (wo) window=dx*255; kernel=dx*1; (wp) window=dx*255; kernel=dx*1; (wq) window=dx*255; kernel=dx*1; (wr) window=dx*255; kernel=dx*1; (ws) window=dx*255; kernel=dx*1; (wt) window=dx*255; kernel=dx*1; (wu) window=dx*255; kernel=dx*1; (wv) window=dx*255; kernel=dx*1; (ww) window=dx*255; kernel=dx*1; (wx) window=dx*255; kernel=dx*1; (wy) window=dx*255; kernel=dx*1; (wz) window=dx*255; kernel=dx*1; (xa) window=dx*255; kernel=dx*1; (xb) window=dx*255; kernel=dx*1; (xc) window=dx*255; kernel=dx*1; (xd) window=dx*255; kernel=dx*1; (xe) window=dx*255; kernel=dx*1; (xf) window=dx*255; kernel=dx*1; (xg) window=dx*255; kernel=dx*1; (xh) window=dx*255; kernel=dx*1; (xi) window=dx*255; kernel=dx*1; (xj) window=dx*255; kernel=dx*1; (xk) window=dx*255; kernel=dx*1; (xl) window=dx*255; kernel=dx*1; (xm) window=dx*255; kernel=dx*1; (xn) window=dx*255; kernel=dx*1; (xo) window=dx*255; kernel=dx*1; (xp) window=dx*255; kernel=dx*1; (xq) window=dx*255; kernel=dx*1; (xr) window=dx*255; kernel=dx*1; (xs) window=dx*255; kernel=dx*1; (xt) window=dx*255; kernel=dx*1; (xu) window=dx*255; kernel=dx*1; (xv) window=dx*255; kernel=dx*1; (xw) window=dx*255; kernel=dx*1; (xx) window=dx*255; kernel=dx*1; (xy) window=dx*255; kernel=dx*1; (xz) window=dx*255; kernel=dx*1; (ya) window=dx*255; kernel=dx*1; (yb) window=dx*255; kernel=dx*1; (yc) window=dx*255; kernel=dx*1; (yd) window=dx*255; kernel=dx*1; (ye) window=dx*255; kernel=dx*1; (yf) window=dx*255; kernel=dx*1; (yg) window=dx*255; kernel=dx*1; (yh) window=dx*255; kernel=dx*1; (yi) window=dx*255; kernel=dx*1; (yj) window=dx*255; kernel=dx*1; (yk) window=dx*255; kernel=dx*1; (yl) window=dx*255; kernel=dx*1; (ym) window=dx*255; kernel=dx*1; (yn) window=dx*255; kernel=dx*1; (yo) window=dx*255; kernel=dx*1; (yp) window=dx*255; kernel=							

06. Kernel-Scoped Bit-Width Optimization

The verification-audit fixes above were correct but not free: LUT usage rose from 1411 to 1605 to buy real correctness (the generic adder tree and the output FIFO both cost fabric).

This phase's goal was to recover that cost and go further, by no longer paying for bit-width the kernel never



Worst-case product $255 \times 8 = 2040$ (13b) | Worst 9-term sum 18,360 => 16-bit accumulator is safe

Figure 10: Kernel Scoped Bit-Width Optimization

Overflow safety proof

Worst-case product = $255 \times 8 = 2,040$ (fits in 13 bits, well under the $4 \times 8 = 32$ -bit naive bound).

Worst-case 9-term accumulation = $9 \times 2,040 = 18,360$, comfortably under the 16-bit signed range (32,768).

No overflow risk is introduced by the narrower accumulator.

The $|coeff| \leq 8$ restriction is hardware-enforced by the narrowed register width in `kernel_memory.v` and `mac_pair.v` - an out-of-range coefficient simply cannot be stored, it is not merely flagged at runtime.

RTL, testbench (`tb_top_2px.v`) and the Python golden model (`prepare_stimulus.py`, `verify.py`) were all updated to the new widths together and re-verified bit-exact before this result was trusted.

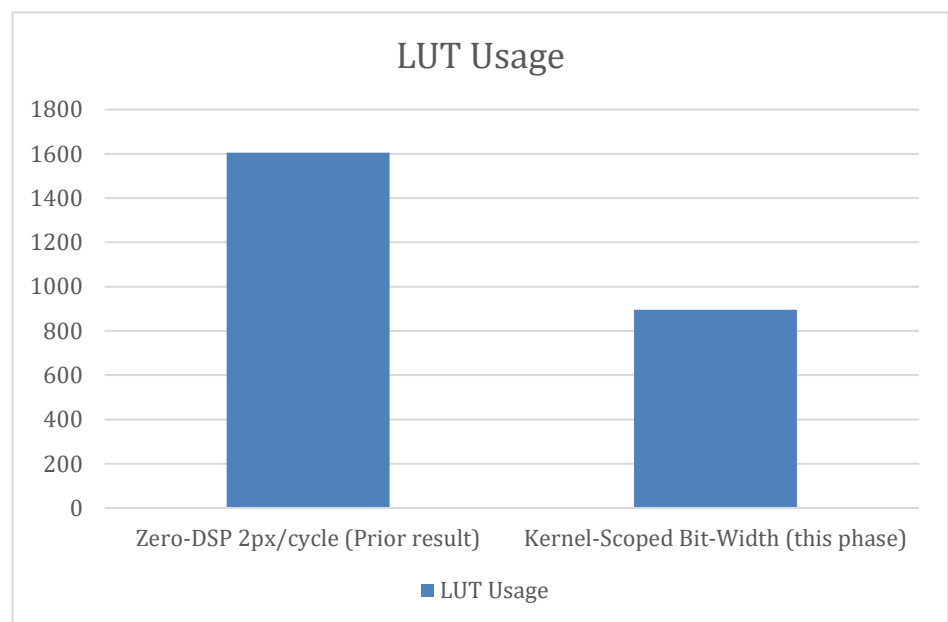


Figure 11: LUT Usage

07. Synthesis and Implementation Results

Metric	Before (1605 LUT)	After (896 LUT)
LUTs	1605	896
Flip-Flops	1288	1049
DSP slices	0	0
BRAM tiles	1	1
Power (total)	0.122 W	0.113 W
WNS (timing slack)	+32.627 ns	+33.802 ns
Figure of Merit	9.61×10^{-3}	1.777×10^{-2}

For reference, the routed utilization report at the 1605-LUT checkpoint breaks the logic down as 1573 LUTs as logic plus 32 as shift-register memory, 1288 flip-flops (529 with async reset, 759 with sync reset), 80 F7 muxes and 40 F8 muxes, and a power split of 0.051 W dynamic against 0.070 W static at 0.122 W total - the same Vivado report_utilization / report_power methodology used to confirm every number in this report.

Throughput
Power x (LUTs + 50xDSPs + 100xBRAMs) With DSPs held at zero and BRAM unchanged at 1 tile across both columns,

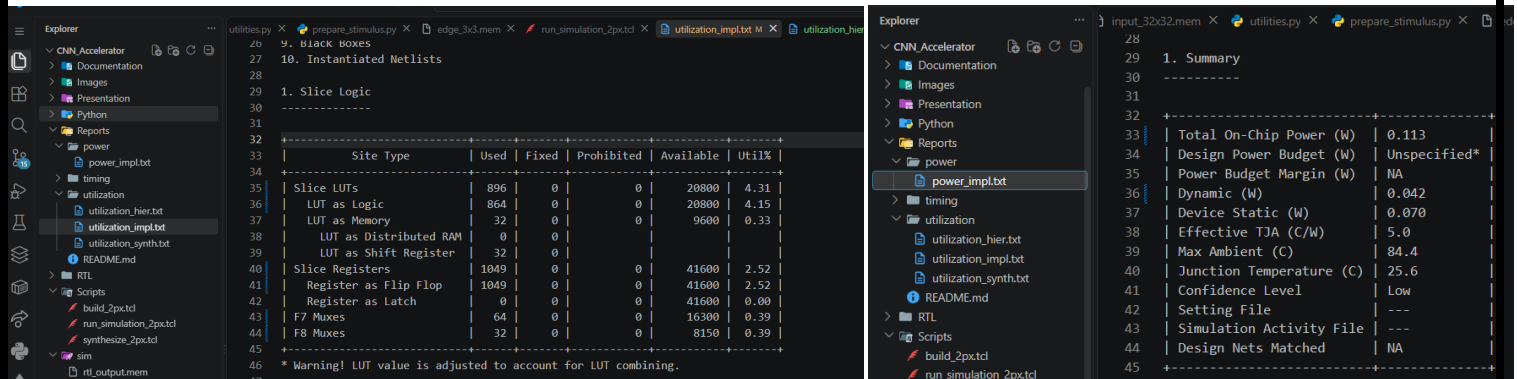


Figure 13: Utilization report

Figure 12: Power Report

the entire FoM gain in this phase comes from the LUT reduction and the small power drop that follows directly from it.

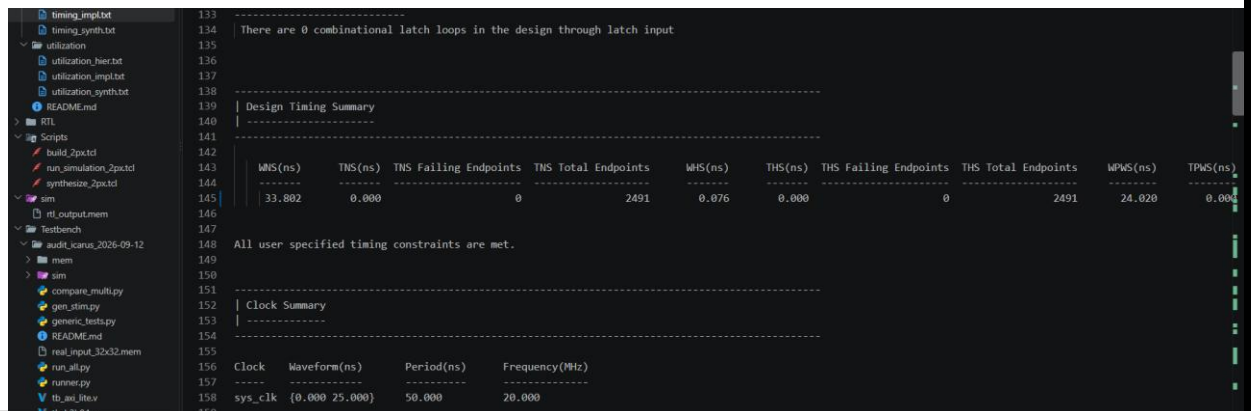


Figure 14: Timing Report

08. Figure of Merit Analysis

Full FoM history, computed from the same formula at every routed checkpoint:

Phase	T (px/cyc)	LUT	DSP	BRAM	Power (W)	FoM
1px Baseline	1	842	0	0.5	0.149	7.520×10^{-3}
MP4 4-DSP (failed timing)	2	738	4	1.0	0.233	8.269×10^{-3}
Zero-DSP 2px (adopted)	2	1411	0	1.0	0.120	1.100×10^{-2}
Verification Audit Fixes	2	1605	0	1.0	0.122	9.610×10^{-3}
Kernel-Scoped Bit-Width (final)	2	896	0	1.0	0.113	1.777×10^{-2}

* The MP4 row never closed timing (WNS -4.338 ns) so its FoM is provisional/invalid and is shown only for completeness, never as a competition result.

The final design improves on the adopted zero-DSP baseline by +62% ($1.10 \times 10^{-2} \rightarrow 1.777 \times 10^{-2}$) and on the immediately prior, verification-hardened checkpoint by +85% ($9.61 \times 10^{-3} \rightarrow 1.777 \times 10^{-2}$), while keeping DSPs at zero and BRAM at exactly 1 tile throughout every point of improvement came from LUTs and the power that follows them.

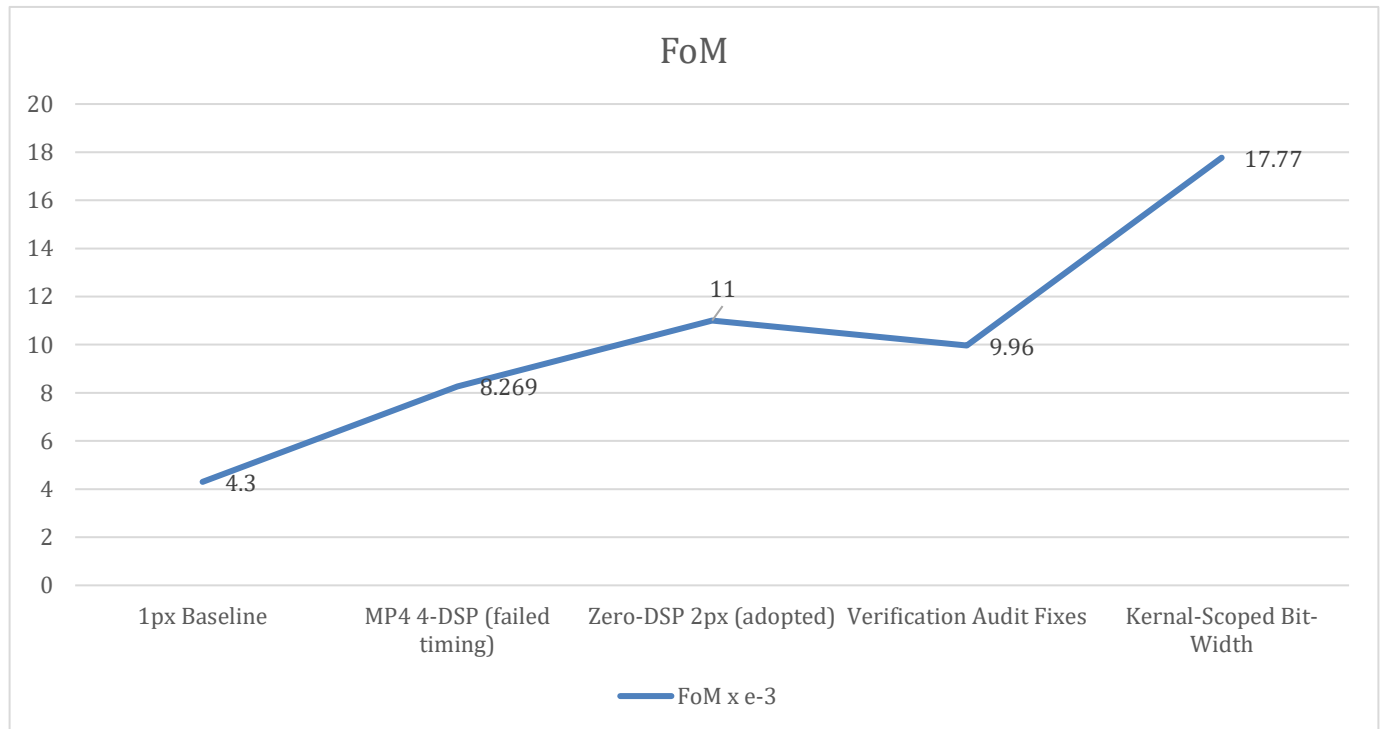


Figure 15: FoM Comparison across time

09. Project Timeline

1px Baseline
842 LUT
FoM $7.52e-3$

**MP4 4-DSP
attempt - FAILED**
WNS -4.338ns

**Zero-DSP 2px
adopted**
1411 LUT

**Verification Audit
bug fixes**
1605 LUT

**Kernel-Scoped
Bit-Width - FINAL**
896 LUT, $1.78e-2$



When	Milestone
Late July 2026	Project scaffolding: folder structure, Python golden model, initial RTL skeleton.
Sep 5	First working 1px/cycle design; fixed an oversized output FIFO and two reset-hazard DRC violations. 842 LUT, FoM 7.52×10^{-3} .
Sep 10 - 11 AM	Built the 2px front end; tried a 4-DSP/120MHz shared-multiplier engine. Verified functionally, but failed routed timing (WNS -4.338 ns) - abandoned.
Sep 11 PM	Adopted the fully-parallel zero-DSP engine instead: same 20MHz clock, no DSPs, no CDC. First timing-closed 2px result: 1411 LUT, FoM 1.10×10^{-2} .
Sep 12 (audit)	46-row verification audit found and fixed two real RTL bugs (generic adder tree, output FIFO); added a from-scratch AXI4-Lite slave. 1605 LUT, FoM 9.61×10^{-3} .
Sep 12 (final)	Kernel-scoped bit-width optimization: $ coeff \leq 8$, Acc_W 16-bit. Final: 896 LUT, FoM 1.777×10^{-3} - best routed result in the project.

10. Requirements Traceability

Every requirement in the competition announcement was checked against a specific file, test result, or report in the repository - not a general claim of compliance. The table below is that mapping: each row names the requirement, and the evidence column points to the exact artifact that proves it, so a reviewer can verify any line independently without re-running the design.

Requirement	Evidence
Min. 32x32 grayscale input	Images/input_32x32.mem, generated by Python/prepare_stimulus.py
Unsigned fixed-point input pixels	8-bit unsigned pixel bus into image_memory_2px.v
Programmable NxN kernel	kernel_memory.v register file, loaded before each run
Kernel precision	coeff <= 8 enforced by 4-bit register width (Sec. 06); within the 8-bit spec ceiling
Stride = 1	window_generator_2px.v advances the window by exactly one column/row per cycle pair
Output precision	16-bit signed accumulator, overflow-proven safe (Sec. 06)
ReLU activation (bonus)	Inline in top_2px.v output stage; testplusarg RELU=1 in Scripts/run_simulation_2px.tcl
Golden-model verification	Python golden_model.py vs. RTL sim, verify.py bit-for-bit compare - 900/900 and 46/46 test-plan rows
Synthesis results	Reports/utilization, Reports/timing, Reports/power - Vivado 2025.1, routed in the Github repo Reports folder
Figure of Merit	1.777×10^{-2} - full derivation in Sec. 08

Section 12 restates this same evidence in the exact Table 1 format specified in the competition announcement, for direct side-by-side comparison against the specification.

11. Verification methods

This project used two independent, cross-checked verification flows rather than one, because a single tool proving itself right is not the same as a design being right. The primary flow is RTL simulation in Vivado XSim (Testbench/tb_top_2px.v) - every synthesis checkpoint in this report was verified against it. A second, tool-independent flow, built in Icarus Verilog (Testbench/audit_icarus_2026-09-12/), exists specifically to run the formal 46-row test plan end-to-end without needing a Vivado session for every row, and to give any single-toolchain blind spot a chance to be caught.

Both flows check against the same source of truth: a bit-accurate Python golden model (golden_model.py, convolution.py, image_loader.py) that computes unsigned-pixel / signed 8-bit-kernel / signed 20-bit-accumulator convolution exactly as the RTL is specified to. verify.py performs the automated RTL-vs-golden comparison and exits 0 (PASS) or 1 (FAIL), so correctness is never a visual judgment call.

Testbench architecture - isolate first, then integrate:

- Unit-level: tb_mac_unit.v, tb_ctrl_unit.v, tb_kernel_unit.v, tb_imgmem_unit.v each drive one RTL module directly, so a failure points at a single file.
- System-level: a reusable harness, tb_generic.v, drives the full top_2px.v datapath with geometry passed in as -P overrides - this is what caught the K=5 adder-tree bug below.
- Multi-phase scenarios: tb_backpressure.v, tb_reload_switch.v, tb_rst04.v, tb_b2b04.v, tb_tim02.v drive sequences a single-frame test can't - reset mid-stream, kernel reload mid-stream, back-to-back frames.
- Bus-functional: tb_axi_lite.v (new, 2026-09-12) drives real AXI4-Lite write/read handshakes against axi_lite_top_2px.v - the only testbench that exercises the register interface in Section 12 rather than the raw ports.

Verification_Test_Plan.xlsx organizes every case into six categories - FUNC (functional correctness), BND (boundary/corner values), RST (reset behavior), B2B (back-to-back operation), ERR (error/edge handling), and TIM (timing/protocol) - 46 rows total. The first full run (2026-09-12) found 38 Passed, 4 Failed, 4 Blocked; every Failed/Blocked row was then either fixed in RTL or resolved as a plan-wording issue and re-verified.

Verification Test Plan — 46/46 Rows Passed, by Category

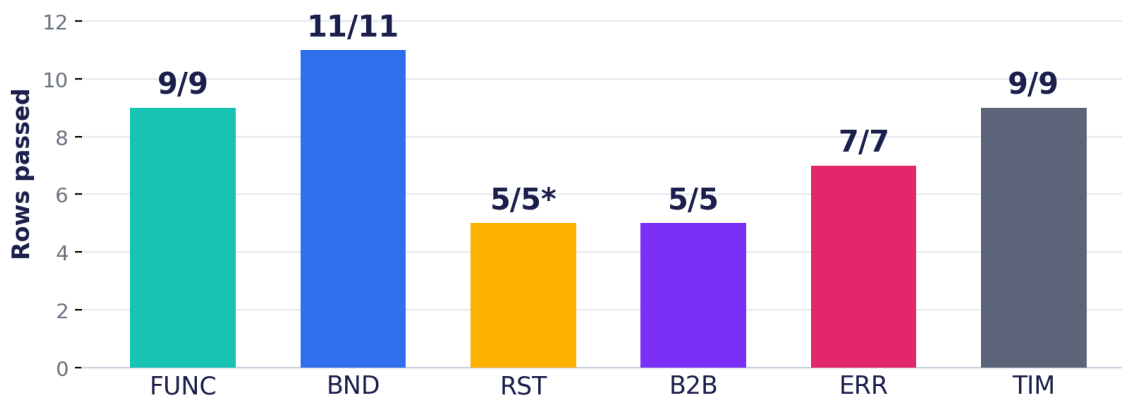


Figure 16: Verification Plan Topics

FPGA CNN-CONVOLUTION ACCELERATOR

The audit earned its keep: it found two real RTL bugs that had passed every earlier test. `mac_pair.v`'s final adder tree was hardcoded to sum exactly 9 product terms, so any kernel size other than $K=3$ silently dropped taps instead of erroring - caught by a $K=5$ case (784/784 wrong outputs, 0 errors reported). `top_2px.v` hard-wired `Out_Full` to 0 with no queue behind it, so a result computed while the consumer wasn't reading was silently overwritten instead of held - caught by `tb_backpressure.v`. Both are fixed and re-verified; Section 12 covers how the fix changed the output path itself.

Two limitations are worth stating plainly. First, the Icarus flow runs in a sandbox without Vivado installed - a genuine second opinion, not a substitute for re-running the official Vivado/XSim simulation against the same RTL before submission. Second, a golden-model PASS only proves the RTL and the golden model agree on whatever stimulus file both were given - it caught nothing when a hand-typed kernel coefficient's sign was mistyped in `prepare_stimulus.py`, because both sides read the same wrong file. That bug only surfaced from a visual check of the rendered output (Section 12); the fix going forward is to generate coefficients programmatically rather than typing them by hand. Third, and still open: RST-03 (reset with results still queued) carries a Passed* because its audit note pre-dates the `fifo_pair.v` fix and has never actually been re-run against the FIFO-equipped design - flagged here rather than quietly counted as clean.

```
PS C:\Users\Xps\Desktop\CNN_Accelerator> python Testbench\audit_icarus_2026-09-12\run_all.py
tb_mac_unit: TB_MAC_UNIT: 22 PASS, 0 FAIL (22 checks scraped)
tb_ctrl_unit: TB_CTRL_UNIT: 24 PASS, 0 FAIL (24 checks scraped)
tb_kernel_unit: TB_KERNEL_UNIT: 5 PASS, 0 FAIL (5 checks scraped)
tb_imgmem_unit: TB_IMGMEM_UNIT: 2 PASS, 0 FAIL (2 checks scraped)

=== Stage 2/4: FIFO backpressure + AXI-Lite testbenches ===
tb_backpressure: TB_BACKPRESSURE: 15 PASS, 0 FAIL (15 checks scraped)
tb_axi_lite: TB_AXI_LITE: 12 PASS, 0 FAIL (12 checks scraped)

=== Stage 3/4: golden-model bit-exact comparisons ===
FUNC-01: 900 outputs, 0 mismatches -> PASS
FUNC-03: 900 outputs, 0 mismatches -> PASS
FUNC-05-seed0: 900 outputs, 0 mismatches -> PASS
FUNC-05-seed1: 900 outputs, 0 mismatches -> PASS
FUNC-05-seed2: 900 outputs, 0 mismatches -> PASS
FUNC-05-seed3: 900 outputs, 0 mismatches -> PASS
FUNC-05-seed4: 900 outputs, 0 mismatches -> PASS
FUNC-05-seed5: 900 outputs, 0 mismatches -> PASS
FUNC-05-seed6: 900 outputs, 0 mismatches -> PASS
FUNC-05-seed7: 900 outputs, 0 mismatches -> PASS
FUNC-05-seed8: 900 outputs, 0 mismatches -> PASS
FUNC-05-seed9: 900 outputs, 0 mismatches -> PASS
FUNC-06: 3844 outputs, 0 mismatches -> PASS
FUNC-07: 784 outputs, 0 mismatches -> PASS
BND-01: 900 outputs, 0 mismatches -> PASS
BND-02: 3844 outputs, 0 mismatches -> PASS
BND-03: 900 outputs, 0 mismatches -> PASS
BND-04: 900 outputs, 0 mismatches -> PASS
BND-05: 900 outputs, 0 mismatches -> PASS
BND-06: 900 outputs, 0 mismatches -> PASS
BND-09: 900 outputs, 0 mismatches -> PASS
BND-10a: 900 outputs, 0 mismatches -> PASS
BND-10b: 900 outputs, 0 mismatches -> PASS
[tb_reload_switch] RUN1: captured 36 (expected 36)
[tb_reload_switch] RUN2: captured 36 (expected 36)
[tb_rst04] RST04_RUN1: captured 36
[tb_rst04] RST04_RUN2: captured 36
[tb_rst04] C:\Users\Xps\Desktop\CNN_Accelerator\Testbench\audit_icarus_2026-09-12\tb_rst04.v:78:
[tb_b2b04] B2B04_RUN0: captured 36
[tb_b2b04] B2B04_RUN1: captured 36
[tb_b2b04] B2B04_RUN2: captured 36
[tb_b2b04] B2B04_FULL_ASSERTED_CYCLES: 0
[tb_b2b04] C:\Users\Xps\Desktop\CNN_Accelerator\Testbench\audit_icarus_2026-09-12\tb_b2b04.v:72:
[tb_tim02] TIM02_LATENCY_CYCLES: 16
[tb_tim02] TIM02: captured 36 (expected 36)
[tb_tim02] C:\Users\Xps\Desktop\CNN_Accelerator\Testbench\audit_icarus_2026-09-12\tb_tim02.v:63:
FUNC-04/B2B-03 run1: 36 outputs, 0 mismatches -> PASS
FUNC-04/B2B-03 run2: 36 outputs, 0 mismatches -> PASS
RST-04 run1: 36 outputs, 0 mismatches -> PASS
RST-04 run2: 36 outputs, 0 mismatches -> PASS
B2B-04 frame0: 36 outputs, 0 mismatches -> PASS
B2B-04 frame1: 36 outputs, 0 mismatches -> PASS
B2B-04 frame2: 36 outputs, 0 mismatches -> PASS
TIM-02 value check: 36 outputs, 0 mismatches -> PASS
RST-03 post-reset clean run: 36 outputs, 0 mismatches -> PASS

=== Stage 4/4: writing results into Verification_Test_Plan.xlsx ===
updated 44 rows, saved Verification_Test_Plan.xlsx
left unchanged (not derivable by this script): FUNC-02, B2B-05
final counts: {'Passed': 45, 'Passed*': 1, 'Failed': 0, 'Blocked': 0, 'Not Run': 0}

=== DONE ===
All 44 simulated rows PASS.
PS C:\Users\Xps\Desktop\CNN_Accelerator>
```

Figure 17: Verification output in VSCode

12. Output generation

This section covers how the accelerator actually produces and exposes a result, at three levels: the hardware output path inside `top_2px.v`, the register interface a host uses to read it, and the simulation-side capture used to check and visualize it during development.

Every cycle, `mac_pair.v` produces two signed 20-bit sums (one per pixel lane), gated through a single ReLU mux per lane (`Relu_En`, applied combinationally, no separate module). Until 2026-09-12 those results went straight to the output ports with no buffering at all - `Out_Full` was hard-wired to 0, so a result produced while the consumer wasn't asserting `Out_Rd_En` was silently overwritten the next cycle instead of held. That is real data loss, not a cosmetic gap, and it is now fixed: a new 16-deep, first-word-fall-through queue, `fifo_pair.v`, sits between `mac_pair.v` and the output ports. `Out_Data0/Out_Data1` are combinational reads of the queue's head entry (so they hold their value until an actual read pops it), `Out_Valid` reflects whether the queue is non-empty, and `Out_Full` genuinely asserts once all 16 slots are used - a write while full is dropped rather than corrupting the queue, verified by `tb_backpressure.v`.

top_2px.v — Hardware Output Path

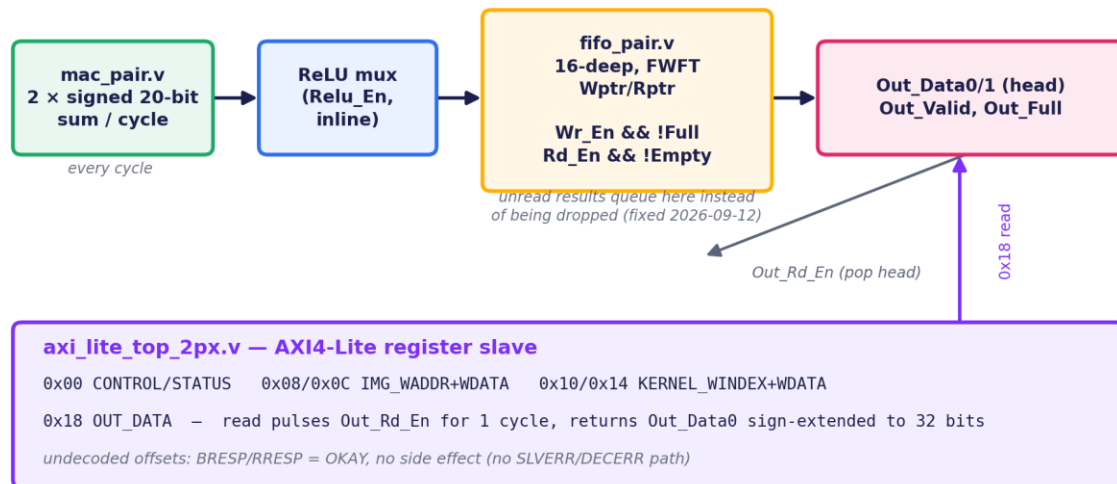


Figure 18: Output path schematic generated by Gemini

Four of the test plan's rows (TIM-08, TIM-09, ERR-06, ERR-07) were written against an “AXI-Lite wrapper” that had no RTL file anywhere in the repository - untestable until `axi_lite_top_2px.v` was written. It is a standard AXI4-Lite slave wrapping `top_2px.v`, using the same `aw_en`-arbitrated write-channel pattern Vivado's own peripheral templates use, with this register map:

Offset	Register	Function
0x00	CONTROL/STATUS	write: bit0 Start (self-clearing pulse), bit1 Relu_En (RW) read: bit0 Busy, bit1 Scan_Done, bit2 Out_Full, bit3 Out_Valid, bit4 Relu_En
0x08	IMG_WADDR	latches the image write address for the next 0x0C write (RW, reads back the latch)
0x0C	IMG_WDATA	pulses <code>Img_We</code> for exactly 1 cycle, using the 0x08 address and this write's low bits as <code>Img_Wdata</code>

FPGA CNN-CONVOLUTION ACCELERATOR

0x10	KERNEL_WINDEX	latches the kernel coefficient index for the next 0x14 write (RW, reads back the latch)
0x14	KERNEL_WDATA	pulses Kernel_We for exactly 1 cycle, using the 0x10 index and this write's low bits as Kernel_Wdata
0x18	OUT_DATA	read: pulses Out_Rd_En for exactly 1 cycle, unconditionally; returns Out_Data0 sign-extended to 32 bits, sampled the cycle of the read

This register map is a new design made to unblock those four rows - no spec for it existed anywhere in the repository before, so it should be checked against any downstream/competition integration expectations rather than assumed. An undecoded offset (e.g. 0x1C) is accepted with BRESP/RRESP = OKAY and no side effect; there is no SLVERR/DECERR path, which is a documented limitation, not an oversight.

For simulation, tb_top_2px.v dumps both result lanes to sim/rtl_output.mem in raster order; verify.py is the actual PASS/FAIL authority, comparing it word-for-word against the golden model. A numeric PASS is not the same as “looks right”, so mem_to_image.py separately renders that same file back into a viewable PNG (Images/rtl_output_preview.png) for a side-by-side visual sanity check against the source image - this is also how the coefficient-sign bug described in Section 11 was actually caught, since the golden-model comparison alone did not.

12.1 Kernal (Horizontal Detection):

[[-1, -2, -1],

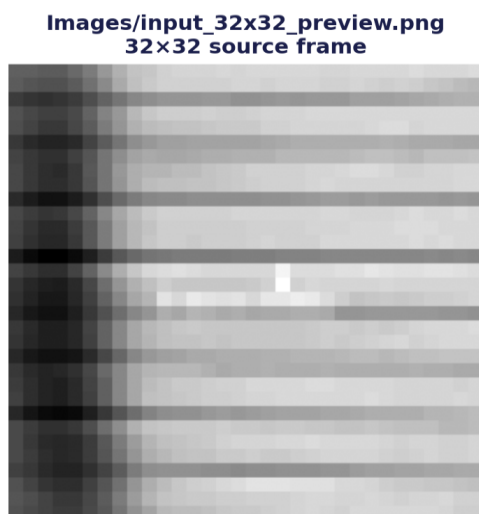
[0, 0, 0],

[1, 2, 1]],



Figure 19: Input image

Real simulation output — generated by mem_to_image.py, not mocked up



sim/rtl_output.mem (dumped by tb_top_2px.v) -> mem_to_image.py -> this PNG. Border pixels are dropped by a valid 3x3 convolution (expected).

Figure 20: Output image

FPGA CNN-CONVOLUTION ACCELERATOR

12.2 Kernal (Vertical Detection):

-1 0 1

-2 0 2

-1 0 1

The kernel is for Vertical Detection, but the image is edging horizontally , so the image predicted is black and it worked, That white column in the output is due to the Vertical shadow in the left of the input image



Figure 21: Input Image

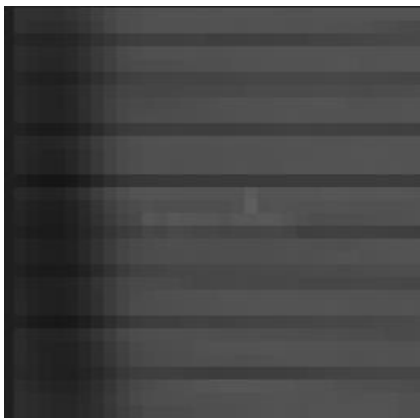


Figure 23: Image 32x32 Input

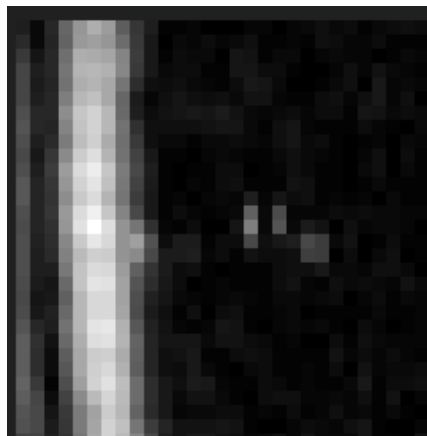


Figure 22: Output Image

12.3 Kernal (Vertical Detection):

-1 0 1

-2 0 2

-1 0 1

The image has vertical borders, so it should provide output image with border detection and it actually succeeded as you can see in the output image as shown.



Figure 24: Input Image

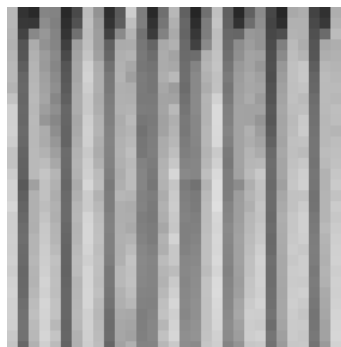


Figure 25: Input Image 32x32

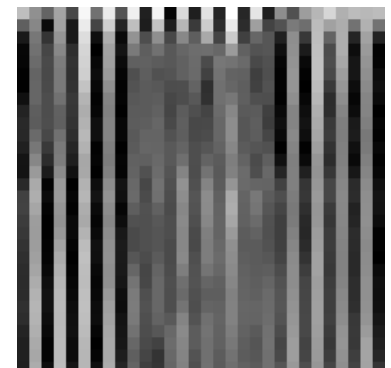


Figure 26: Output Image 32x32

FPGA CNN-CONVOLUTION ACCELERATOR

TCL Output:

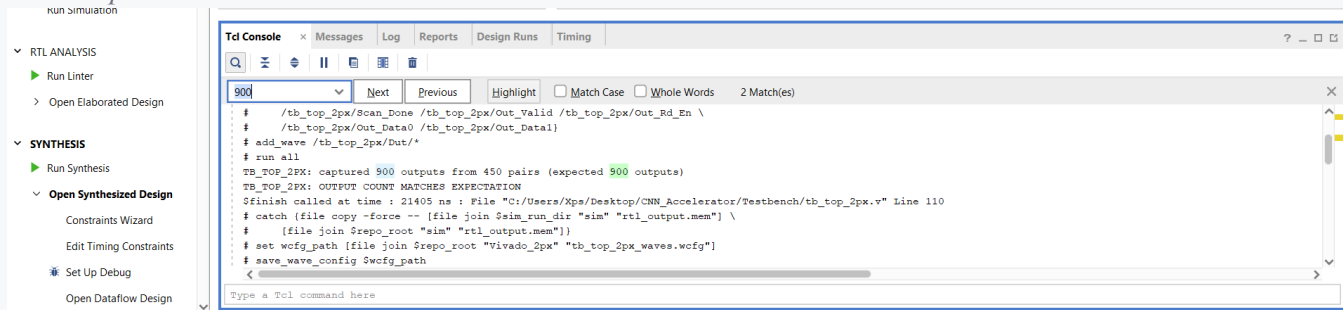


Figure 27: TCL Output

VsCode Terminal:

```
PS C:\Users\Xps\Desktop\CNN_Accelerator>
PS C:\Users\Xps\Desktop\CNN_Accelerator> python Python/prepare_stimulus.py
Wrote Images/input_32x32.mem (32x32, 8-bit unsigned)
Wrote Images/kernels/edge_3x3.mem (vertical Sobel kernel, 3x3 signed 8-bit)
Wrote Images/input_32x32_preview.png (what the RTL will actually process)
PS C:\Users\Xps\Desktop\CNN_Accelerator> python Python\verify.py
Compared 900 output values.
RESULT: PASS - RTL output matches the golden model exactly.
PS C:\Users\Xps\Desktop\CNN_Accelerator> python Python/mem_to_image.py
Read sim/rtl_output.mem (30x30, signed 16-bit)
Wrote Images/rtl_output_preview.png
PS C:\Users\Xps\Desktop\CNN_Accelerator>
```

Figure 28: Vs Code Output Terminal

I usually use about 5 lines to test my system every time I make an edit:

1. **python Python/prepare_stimulus.py** // for generating the mem kernel and image32x32 memory files and generating the same image again in the 32x32 size to make sure it really didn't make any issue

1. **cd {C:/Users/Xps/Desktop/CNN_Accelerator}**
2. **source Scripts/run_simulation_2px.tcl** # for running my design to generate the output memory file
3. **open_wave_database**
./Vivado_2px/CNN_Accelerator_2px.sim/sim_1/behav/xsim/tb_top_2px_behav.wdb
source Scripts/synthesize_2px.tcl # it is just for opening the wave saved by the last simulation

1. **python Python\verify.py** // It is for comparing the output memory file with the memory file outputted from the golden python model
2. **python Python/mem_to_image.py** // It is for generating the output image32x32 again from the output memory
3. **python Testbench/audit_icarus_2026-09-12/run_all.py** //It is for implementation of the verification plan using Icarus Verilog on VSCode

13. Team and Repository

Silicon Minds started as a 3-person team, but the other two members left the project at the very beginning of the competition.

Basel Shrief - sole remaining team member

Every part of this project - the verification plan, the RTL design and every optimization decision, and the Python golden-model / stimulus tooling - was built and carried through entirely by Basel, using AI tools (Claude) as an assistant for code editing, debugging, and documentation.

- RTL design and micro-architecture (2px zero-DSP datapath, kernel-scoped bit-width optimization) - Basel
- Verification plan, testbenches, and Python golden-model checking - Basel
- Synthesis, timing closure, power analysis, and reporting - Basel
- AI-assisted tooling used throughout for implementation speed and documentation

Repository: github.com/basel-sh/CNN_Accelerator

Two branches: master (stable) and aggressive-2px-fom (the optimization work in this report). Design-space history is kept in Documentation/Aggressive2px.md rather than deleted, including the abandoned MP4/MP3 attempts, so the reasoning behind every discarded design stays reviewable.

14. Competition Required Results Table

Table 1, exactly as specified in the 2026 SSCS Egypt Competition Announcement, filled in with this team's routed, verified results.

Parameter	Specification	Team Result	Units	Comments
Input image size	Min. 32x32 grayscale	32 x 32	pixels	Images/input_32x32.mem
Input precision	Unsigned fixed-point	Unsigned, 8-bit fixed-point	bits	Meets spec
Kernel precision	8-bit signed, programmable NxN	4-bit signed, coeff <= 8	bits	Narrowed in-range, within 8-bit ceiling (Sec. 06)
Architecture type	Open (designer's choice)	Fully-parallel, zero-DSP, 2-lane streaming MAC	-	No DSPs, no CDC, single clock
Multipliers / MACs	Open (designer's choice)	18 fabric multipliers (9/lane x 2 lanes)	count	0 DSP48 slices used
Pipeline stages	Open (designer's choice)	K-1 row line-buffer fill + window/MAC/ReLU register stages	-	See Sec. 03 for the full data-flow breakdown
Latency	Open (designer's choice)	~64 cycles fill (2 rows @ 32px, K=3) + fixed pipeline delay	cycles	Estimated from line-buffer depth; then 2 px/cycle steady-state
Throughput	Open (designer's choice)	2	px/cycle	Sustained, no stalls after fill
FPGA utilization	Open (designer's choice)	LUT 4.31% (896/20800), FF 2.52% (1049/41600), BRAM 2.0% (1/50), DSP 0% (0/90)	%	xc7a35tcpg236-1 totals
Maximum frequency	Open (designer's choice)	20 MHz operating; ~62 MHz est. fMax from WNS	MHz	WNS +33.802 ns @ 50 ns period
Power estimate	Open (designer's choice)	0.113	W	Vivado report_power, routed
Verification status	Verified vs. golden model	900/900 bit-exact; 46/46 test-plan rows Passed	-	Python golden model + Icarus Verilog regression
FOM	Higher is better	1.777×10^{-2}	-	+85% vs. immediately prior checkpoint (Sec. 08)